



50

MEDIUM & HARD

SQL INTERVIEW QUESTIONS



Real interview questions with concise answers for
Data Analysts, Data Engineers,
and **Backend Developers.**



QUERY
EXECUTION



JOINS



WINDOW
FUNCTIONS



AGGREGATIONS



CTES &
SUBQUERIES



INDEXES



QUERY
OPTIMIZATION



TRANSACTIONS



REAL
INTERVIEW
PROBLEMS

Vansh Budhiraja

Data • AI • Tech

Swipe →



SQL EXECUTION ORDER

The order in which SQL processes a query.



01

What is the execution order of a SQL query?

- ✓ FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT / OFFSET.

02

Why can't we use column aliases in WHERE clause?

- ✓ WHERE is evaluated before SELECT, so aliases don't exist yet.

03

What is the difference between WHERE and HAVING?

- ✓ WHERE filters rows before grouping. HAVING filters groups after aggregation.

04

Why is execution order important?

- ✓ It helps write correct queries, avoid errors and improve performance.

EXAMPLE

```
SELECT dept, COUNT(*) AS total
FROM employees
WHERE salary > 50000
GROUP BY dept
HAVING COUNT(*) > 2
ORDER BY total DESC
LIMIT 5;
```

→ Query runs in this order, not the order it is written.



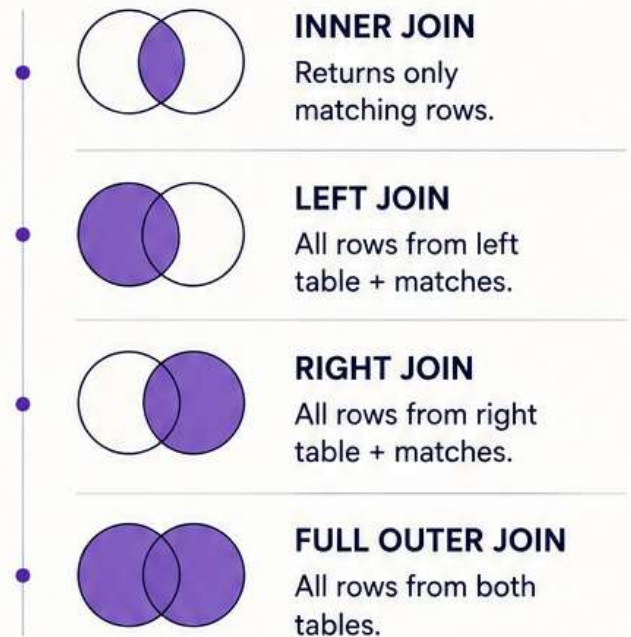
INTERVIEW TIP

Always remember the execution order. It's a top interview favorite!



SQL JOINS

Combine rows from two or more tables.

**01**

What is the difference between INNER JOIN and LEFT JOIN?

- ✓ INNER JOIN returns only matching rows. LEFT JOIN returns all rows from the left table + matches.

02

When should you use FULL OUTER JOIN?

- ✓ Use it when you need all rows from both tables, whether they match or not.

03

How does LEFT JOIN handle unmatched rows?

- ✓ It returns NULL for columns from the right table when there is no match.

04

Can we use JOIN without ON clause?

- ✓ Yes, but it creates a Cartesian product, which is usually not desired.

EXAMPLE

Find customers and their orders (if any) using LEFT JOIN.

```
SELECT c.customer_id, c.name, o.order_id
FROM   customers c
LEFT JOIN orders o
ON     c.customer_id = o.customer_id;
```

→ All customers will be shown, even if they have no orders.



INTERVIEW TIP

Always clarify the business requirement before choosing the type of JOIN.



SQL WINDOW FUNCTIONS

Perform calculations across a set of rows related to the current row.



RANK()

Ranks rows with gaps for ties.



DENSE_RANK()

Ranks rows without gaps for ties.



ROW_NUMBER()

Assigns a unique number to each row.



NTILE(n)

Divides rows into n buckets.

01

What is the difference between **RANK()** and **DENSE_RANK()**?



RANK() skips numbers after ties. **DENSE_RANK()** doesn't skip.

02

When should you use **ROW_NUMBER()**?



When you need a unique number for every row, for example, for deduplication.

03

What does **NTILE(n)** do?



It splits the result set into n equal parts as much as possible.

04

Can window functions be used with **GROUP BY**?



Yes, but window functions are calculated after **GROUP BY**. You cannot use them in **WHERE**.

EXAMPLE

Get rank and row number of employees by salary in each department.

```
SELECT dept_id, emp_id, salary,
       RANK() OVER (PARTITION BY dept_id
                    ORDER BY salary DESC) AS rnk,
       ROW_NUMBER() OVER (PARTITION BY dept_id
                           ORDER BY salary DESC) AS rn
FROM employees;
```

→ Partition by dept,
then rank and
number by
salary.



INTERVIEW TIP

Always pay attention to **PARTITION BY** and **ORDER BY** inside **OVER()**. It changes the result completely.



SQL AGGREGATIONS

Perform calculations on a set of rows and return a single result.

COUNT()**COUNT()**

Counts the number of rows.

SUM()**SUM()**

Returns the total sum of values.

AVG()**AVG()**

Returns the average of values.

MIN()**MIN()**

Returns the smallest value.

MAX()**MAX()**

Returns the largest value.

01

What is the difference between **COUNT(*)** and **COUNT(column)**?



COUNT(*) counts all rows, including those with NULLs. **COUNT(column)** counts only non-NULL values.

02

Does **AVG()** ignore NULL values?



Yes, **AVG()** ignores NULL values while calculating the average.

03

Can we use multiple aggregate functions in the same query?



Yes, you can use multiple aggregate functions together in the **SELECT** clause.

04

Can we use aggregate functions in **WHERE**?



No, aggregate functions are computed after **WHERE**. Use **HAVING** clause instead.

EXAMPLE

Find total salary and average salary by department.

```
SELECT dept_id,  
       COUNT(*) AS total_employees,  
       SUM(salary) AS total_salary,  
       AVG(salary) AS avg_salary,  
       MIN(salary) AS min_salary,  
       MAX(salary) AS max_salary  
FROM employees  
GROUP BY dept_id;
```

*GROUP BY
is mandatory
when using
aggregates
with columns.*



INTERVIEW TIP

Know how and when aggregates work. Be clear about NULLs, **GROUP BY** and **HAVING** clause.



SQL CTEs & SUBQUERIES

Break down complex queries into logical parts or use nested queries.

CTE (Common Table Expression)



Defined using WITH clause.



Improves readability and maintainability.



Can be referenced multiple times in the main query.



Better for complex, multi-step logic.

SUBQUERY



Nested inside another query.



Useful for one-time calculations or filters.



Executed each time it is encountered.



Better for simple conditions.

01

What is the main advantage of using a CTE?



It improves readability, allows code reuse, and makes complex queries easier to understand.

02

When should you use a subquery instead of a CTE?



Use a subquery for simple, one-off calculations or when the result is needed in a specific clause.

03

Can a CTE be recursive?



Yes, recursive CTEs can reference themselves to solve hierarchical or iterative problems.

04

Where can a subquery be used?



In SELECT, FROM, WHERE, and HAVING clauses.

EXAMPLE

CTE Example: Find employees with salary above department average salary.

```
WITH dept_avg AS (  
  SELECT dept_id, AVG(salary) AS avg_sal  
  FROM employees  
  GROUP BY dept_id  
)  
SELECT e.name, e.salary, e.dept_id  
FROM employees e  
JOIN dept_avg d ON e.dept_id = d.dept_id  
WHERE e.salary > d.avg_sal;
```

CTE makes the query easier to read and avoids repeating the subquery.



INTERVIEW TIP

CTEs are not always faster than subqueries. Performance depends on the database optimizer and the complexity of the query.



SQL GROUP BY VS HAVING

Understand the key difference and when to use each clause.

GROUP BY	HAVING
 Groups rows with same values.	 Filters groups after aggregation.
 Used with aggregate functions.	 Used with aggregate functions.
 Filters before aggregation.	 Filters after aggregation.
 Appears after WHERE and before HAVING.	 Appears after GROUP BY and before ORDER BY.
 Cannot use aggregate functions in WHERE.	 Can use aggregate functions here.

01 Why can't we use aggregate functions in WHERE?

- ✓ Because WHERE is evaluated before aggregation. Aggregate functions are calculated after GROUP BY.

02 When should you use HAVING instead of WHERE?

- ✓ When you need to filter groups based on aggregate conditions, such as COUNT(), SUM(), AVG(), etc.

03 Can we use HAVING without GROUP BY?

- ✓ Yes, but it treats the entire result set as a single group.

04 What happens if we don't use GROUP BY with aggregates?

- ✓ The entire result is treated as one group, and aggregates return a single value.

EXAMPLE

Find departments with more than 5 employees.

```
SELECT dept_id,  
       COUNT(*) AS total_employees  
FROM   employees  
GROUP BY dept_id  
HAVING COUNT(*) > 5  
ORDER BY total_employees DESC;
```

WHERE filters rows.
HAVING filters groups.



INTERVIEW TIP

Think in this order:
WHERE filters rows,
GROUP BY groups them,
HAVING filters those groups.



SQL ORDER BY

Sort the result set
in ascending or
descending order.



ASC (Ascending)

Sorts in ascending order
(default).



DESC (Descending)

Sorts in descending order.



MULTIPLE COLUMNS

Sort by more than one
column.



NULLS FIRST / LAST

Controls the position of
NULL values.



ALIAS IN ORDER BY

You can use column aliases
defined in SELECT.

01

Is ORDER BY mandatory
in a SELECT query?



No, ORDER BY is optional. Without it,
the order of rows is not guaranteed.

02

Can we use column alias
in ORDER BY?



Yes, you can use the alias defined
in the SELECT clause.

03

What is the default order
of NULL values?



It depends on the database.
Use NULLS FIRST or NULLS LAST
to control it.

04

Can we use ORDER BY
with DISTINCT?



Yes, ORDER BY works with DISTINCT
to sort the unique results.

EXAMPLE

Display employees ordered by salary
(highest first), then by name (A to Z).

```
SELECT emp_id, emp_name, salary
FROM employees
ORDER BY salary DESC, emp_name ASC;
```

First sort by
salary (high
to low), then
by name
(A to Z).
→



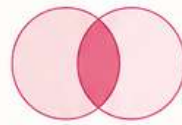
INTERVIEW TIP

Always be explicit with
ORDER BY. It ensures
consistent results and
avoids confusion.



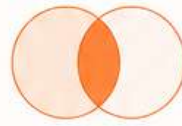
SQL JOINS

Combine rows from two or more tables based on a related column.



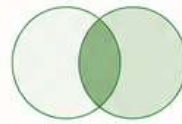
INNER JOIN

Returns rows that have matching values in both tables.



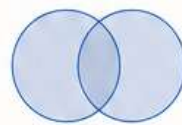
LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table and matching rows from the right table.



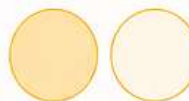
RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and matching rows from the left table.



FULL OUTER JOIN

Returns all rows when there is a match in either left or right table.



CROSS JOIN

Returns the Cartesian product of both tables.

01

Which join returns only matching rows?



INNER JOIN returns rows that have matching values in both tables.

02

Which join returns all rows from the left table?



LEFT JOIN returns all rows from the left table and matching rows from the right table.

03

Which join returns all rows when there is no match?



FULL OUTER JOIN returns all rows when there is a match in either left or right table.

04

What happens in a CROSS JOIN?



It returns the Cartesian product of both tables. Each row from the first table is paired with every row of the second table.



EXAMPLE

List all orders with customer names (including orders without a customer).

```
SELECT o.order_id, o.amount, c.customer_name
FROM   Orders o
LEFT JOIN Customers c
ON o.customer_id = c.customer_id;
```

LEFT JOIN keeps all orders even if customer is missing.



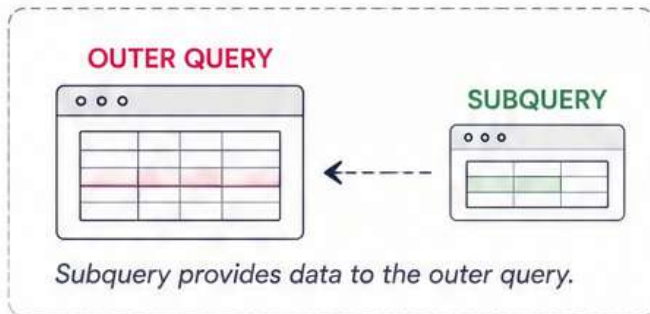
INTERVIEW TIP

Understand the data and the requirement before choosing a join. The wrong join can lead to missing or duplicated data.



SQL SUBQUERIES

A query inside another query, used to retrieve data for the outer query.



TYPES OF SUBQUERIES

1



SCALAR SUBQUERY

Returns a single value.
Used with =, <, >, <=, >=, <>.

2



MULTI-ROW SUBQUERY

Returns multiple rows.
Used with IN, NOT IN, ANY, ALL.

3



MULTI-COLUMN SUBQUERY

Returns multiple columns.
Used with IN (column1, column2).

4



CORRELATED SUBQUERY

Depends on the outer query.
Executed once for each row of the outer query.

5



EXISTS / NOT EXISTS

Checks the existence of rows.
Returns TRUE or FALSE.

01

When should you use a subquery?



Use a subquery when the solution requires data that depends on the result of another query.

02

What is the difference between IN and EXISTS?



IN compares values, while EXISTS checks if a subquery returns at least one row.

03

Can a subquery be used in the FROM clause?



Yes, it can be used in the FROM clause as a derived table (inline view).

04

Are correlated subqueries less efficient?



They can be slower on large datasets because the subquery runs once for each row of the outer query.

EXAMPLE

Find employees who earn more than their department average salary.

```
SELECT e.employee_id, e.name, e.salary
FROM employees e
WHERE e.salary > (
  SELECT AVG(salary)
  FROM employees
  WHERE department_id = e.department_id
);
```

Correlated subquery:
Uses value from the outer query.



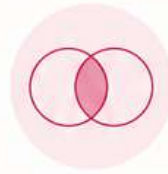
INTERVIEW TIP

Always check if a subquery can be replaced with a **JOIN** for better performance. Understand the data and requirement before choosing the approach.



SQL SET OPERATORS

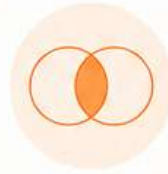
Combine the result sets of two or more **SELECT** statements.



UNION

Returns all distinct rows from both queries. (Removes duplicates)

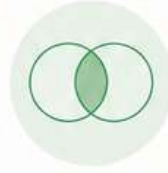
Default



UNION ALL

Returns all rows from both queries, including duplicates.

Faster



INTERSECT

Returns only the rows that are common in both queries.

Common rows



EXCEPT (MINUS)

Returns rows from the first query that are not in the second.

Left minus Right



EXCEPT ALL

Returns all rows from the first query that are not in the second, including duplicates.

Keeps duplicates

01

What is the difference between **UNION** and **UNION ALL**?



UNION removes duplicates and returns distinct rows. **UNION ALL** includes all rows, including duplicates.

02

Is **UNION** faster than **UNION ALL**?



No, **UNION ALL** is faster because it does not check for duplicates.

03

Do the **SELECT** statements in a set operator need to be the same?



Yes, both **SELECT** statements must have the same number of columns and compatible data types.

04

Can we use **ORDER BY** in each **SELECT** of a set operator?



No, **ORDER BY** can only be used at the end of the complete set operation.



EXAMPLE

List all unique employee IDs from full-time and part-time employees.

```
SELECT employee_id FROM full_time_employees
UNION
SELECT employee_id FROM part_time_employees
ORDER BY employee_id;
```

UNION returns unique employee IDs from both tables.



INTERVIEW TIP

Always make sure the number of columns and their data types match. Know which operator fits the requirement:

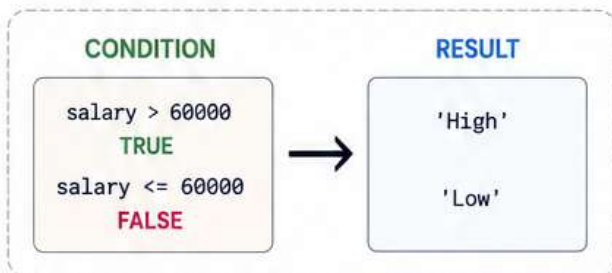
UNION for unique, **UNION ALL** for performance, **INTERSECT** for common, **EXCEPT** for differences.



SQL

CONDITIONAL EXPRESSIONS

Perform different actions based on conditions. Used to add logic to queries.



01

When should you use CASE instead of IF?



Use CASE WHEN when you have multiple conditions to evaluate.

02

What does COALESCE() do?



It returns the first non-NULL value from a list of expressions.

03

How is NULLIF() useful?



It helps prevent division by zero and returns NULL when two expressions are equal.

04

Are these functions portable across databases?



CASE WHEN, COALESCE(), and NULLIF() are ANSI standard and widely supported. Others may vary by database.



EXAMPLE

Categorize employees based on salary.

```
SELECT employee_id, name, salary,
       CASE
         WHEN salary >= 80000 THEN 'High'
         WHEN salary >= 50000 THEN 'Medium'
         ELSE 'Low'
       END AS salary_category
FROM employees;
```

Evaluate conditions in order. The first true condition determines the result.



INTERVIEW TIP

Use **conditional** expressions to make your queries **smarter** and your results more meaningful. Always handle **NULLs** carefully!

CASE
WHEN
THEN
ELSE
END

CASE WHEN

Evaluates conditions and returns a result when the first condition is true.

IF()

IF()

Returns one value if a condition is true, and another if it is false.
(MySQL)

IIF()

IIF()

Shorthand for IF().
(SQL Server)

COALESCE()

COALESCE()

Returns the first non-NULL value in a list of expressions.

NULLIF()

NULLIF()

Returns NULL if two expressions are equal, otherwise returns the first expression.

...

Others

CHOOSE(), DECODE(), IFNULL(), ISNULL() and more.



SQL AGGREGATE FUNCTIONS

Perform calculations on a set of values and return a single result.

EMPLOYEES TABLE (Sample)

emp_id	name	department	salary
1	Alice	HR	50000
2	Bob	HR	60000
3	Charlie	IT	80000
4	David	IT	90000
5	Eva	Sales	45000

COUNT(*)



COUNT()

Returns the number of rows.
COUNT(column) counts non-NULL values in that column.

SUM()



SUM()

Returns the total sum of numeric values.

AVG()



AVG()

Returns the average of numeric values.

MIN()



MIN()

Returns the smallest value in the set.

MAX()



MAX()

Returns the largest value in the set.

Others



Others

STDDEV(), VARIANCE(), GROUP_CONCAT(), etc. for advanced aggregations.

01

What is the difference between COUNT(*) and COUNT(column)?



COUNT(*) counts all rows, including those with NULLs. COUNT(column) counts only non-NULL values.

02

Can aggregate functions be used with WHERE clause?



Yes, aggregate functions can be used with WHERE clause. They are applied after filtering the rows.

03

Can we use aggregate functions without GROUP BY?



Yes, if GROUP BY is not used, aggregate functions are applied to the entire result set and return a single value.

04

Why do we use GROUP BY with aggregate functions?



GROUP BY groups rows that have the same values in specified columns so aggregate functions can be applied to each group.



EXAMPLE

Find the total salary and average salary of employees in each department.

```
SELECT department,  
       COUNT(*) AS total_employees,  
       SUM(salary) AS total_salary,  
       AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department;
```

GROUP BY
department
groups the rows
and aggregates
are calculated
for each group.



INTERVIEW TIP

Understand the data and the question before choosing aggregate functions. Always handle NULLs and use **GROUP BY** when you need results per group.



SQL GROUP BY & HAVING

GROUP BY groups rows that have the same values in specified columns. HAVING filters groups based on conditions.

EMPLOYEES TABLE (Sample)

emp_id	name	department	salary
1	Alice	HR	50000
2	Bob	HR	60000
3	Charlie	IT	80000
4	David	IT	90000
5	Eva	Sales	45000
6	Frank	Sales	55000

GROUP BY
department



department	count(*)
HR	2
IT	2
Sales	2

Groups rows by department.



GROUP BY

Groups rows that have the same values in one or more columns.



HAVING

Filters groups based on aggregate conditions (COUNT, SUM, AVG, etc.).



GROUP BY vs WHERE

WHERE filters rows before grouping.
HAVING filters groups after grouping.



Multiple Columns

GROUP BY can be applied on one or more columns.



Aggregate Functions

HAVING works with aggregate functions and cannot be used with non-aggregate columns (unless in GROUP BY).



Execution Order

FROM → WHERE → GROUP BY →
HAVING → SELECT → ORDER BY

01

What is the difference between WHERE and HAVING?



WHERE filters individual rows before grouping.
HAVING filters groups after aggregation.

02

Can we use HAVING without GROUP BY?



No, HAVING is used only with GROUP BY because it filters groups.

03

Can we use column alias in HAVING?



Yes, most databases (like MySQL) allow using column alias in HAVING.
(But standard SQL does not.)

04

What happens if we use non-aggregated column in SELECT with GROUP BY?



It will cause an error unless that column is included in GROUP BY or used inside an aggregate function.



EXAMPLE

Find departments where total salary is more than 100000.

```
SELECT department,  
       SUM(salary) AS total_salary  
FROM employees  
GROUP BY department  
HAVING SUM(salary) > 100000  
ORDER BY total_salary DESC;
```

HAVING filters the grouped results based on condition.



INTERVIEW TIP

Use **GROUP BY** to organize data into meaningful groups.
Use **HAVING** to filter those groups based on aggregate conditions.
Always remember the order of **execution**!



SQL PRACTICE & TIPS

Consistent practice + strong fundamentals = SQL mastery.

COMPLETE SQL JOURNEY



KEY TAKEAWAYS



Understand the purpose of each clause and function.



Write simple queries first, then optimize.



Always think about the data and the expected output.



Practice different question types regularly.



Focus on accuracy, not just speed.



Review, refactor and keep improving your queries.

01

How can I improve my SQL skills?



Solve problems daily, analyze others' solutions and write queries in different ways.

02

Is SQL enough for data roles?



SQL is essential. Combine it with data analysis, visualization and domain knowledge.

03

What should I do if I get stuck in a question?



Break the problem into smaller parts, understand the data and build step by step.

04

How to write optimized SQL queries?



Use proper indexes, avoid SELECT *, filter early and aggregate only when needed.



EXAMPLE CHALLENGE

Write a query to find the top 3 departments by average salary.

```
SELECT department,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC
LIMIT 3;
```

Practice real problems. That's how you get better!



FINAL NOTE

SQL is not just about syntax, it's about **solving business problems** with data. Keep practicing, stay curious and you'll crack it!